

Materia: Base de Datos II – UTN

Trabajo Práctico: Laboratorio N° 2 – Concurrencia e IA como motor primario

Alumno: Facundo Deseff

Repositorio de GitHub:

<https://github.com/FacundoDeseff/tp2-base-datos.git>

Parte 0 — Puesta a Punto y Protocolo de Seguridad

Protocolo de Seguridad (Regla de 3 Pasos)

Este protocolo se aplica obligatoriamente antes de ejecutar cualquier script generado por un agente de IA sobre la base de datos.

1. Copia

* **Qué significa:** Se trabaja sobre una base de desarrollo, nunca sobre la que contiene datos que importan.

* **Comando concreto:** createdb -T pizzeria_test pizzeria_trabajo

* **Cuándo se salta:** Nunca. Siempre hay copia.

2. Transacción

* **Qué significa:** Todo script que escribe corre primero dentro de BEGIN; y ROLLBACK; para inspeccionar el efecto antes de confirmar nada.

* **Comando concreto:**

BEGIN;

-- [Script de IA acá]

ROLLBACK; -- (O COMMIT si todo está bien)

* **Cuándo se salta:** Nunca. Siempre BEGIN antes de escribir.

3. Respaldo

* **Qué significa:** Se realiza un pg_dump de la copia de trabajo antes de aplicar un cambio estructural (ALTER, DROP, migración).

* **Comando concreto:** pg_dump -Fc pizzeria_trabajo > backup.dump

* **Cuándo se salta:** Nunca. Siempre respaldo antes de DDL.

Parte 1 — Integridad Versionada

```
CREATE TABLE categoria (  
    id_categoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE producto (  
    id_producto BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nombre VARCHAR(150) NOT NULL,  
    precio_lista NUMERIC (10,2) NOT NULL,  
    stock INTEGER NOT NULL DEFAULT 0,  
    id_categoria BIGINT NOT NULL  
);
```

```
ALTER TABLE producto  
    ADD CONSTRAINT ck_producto_precio_stock  
    CHECK (precio_lista > 0 AND stock >= 0);  
  
ALTER TABLE producto  
    ADD CONSTRAINT fk_producto_categoria  
    FOREIGN KEY (id_categoria)  
    REFERENCES categoria (id_categoria)  
    ON DELETE RESTRICT;
```

Declaración de Uso de IA (DUIA) - Parte 1

Campo	Completar
Herramienta	Kiro Agent (Integrado en VS Code)
Spec o prompt utilizado	"Analizá schema.sql. Necesito generar un archivo restricciones.sql para PostgreSQL con las siguientes reglas de negocio: 1. Restricción CHECK en producto para que precio_lista > 0 y stock >= 0. 2. Clave foránea fk_producto_categoria en producto referenciando a categoria(id_categoria) con ON DELETE RESTRICT."
Qué generó	Archivo restricciones.sql con sentencias ALTER TABLE para incorporar constraints chk_producto_precio_stock y fk_producto_categoria.
Qué se aceptó	La totalidad de la sintaxis ALTER TABLE, la combinación del CHECK de precio/stock y el ON DELETE RESTRICT.
Qué se modificó o descartó	Se validó que el CHECK de precio_lista mantenga la condición de ser estrictamente positivo (> 0).
Verificación realizada	"Se ejecutó el script dentro de una transacción. Se probó un INSERT válido que fue aceptado, y un INSERT inválido con precio negativo (-100) que fue rechazado correctamente por el motor con el error 23514 por violar la restricción check."

Parte 2 — Laboratorio de Concurrency

Informe de Laboratorio: Concurrency y Aislamiento - Parte 2

Escenario 1: Espera por Bloqueo (Row-Level Locking con SELECT FOR UPDATE)

* Cómo se reprodujo:

- Sesión A:

BEGIN;

SELECT * FROM producto WHERE id_producto = 1 FOR UPDATE;

(Mantiene la transacción abierta sin COMMIT)

- Sesión B:

BEGIN;

UPDATE producto SET stock = stock - 1 WHERE id_producto = 1;

* Qué se observó: La Sesión B quedó bloqueada en espera activa (hang). Al ejecutar COMMIT en la Sesión A, la Sesión B se desbloqueó inmediatamente y completó su UPDATE.

* Explicación de la IA: SELECT ... FOR UPDATE adquiere un bloqueo exclusivo a nivel de fila (ExclusiveLock de tupla). Cualquier otra transacción que intente modificar la misma fila es puesta en cola hasta que la transacción bloqueante libere el recurso con COMMIT o ROLLBACK.

* Verificación en el motor: Se verificó consultando pg_locks durante la espera, confirmando el estado granted = false para el PID de la Sesión B.

* Conclusión: Confirmado en el motor. Es el mecanismo estándar para evitar Race Conditions en inventario.

Escenario 2: Lectura No Repetible (Non-Repeatable Read)

* Cómo se reprodujo:

- Sesión A (Read Committed por defecto):

BEGIN;

SELECT precio_lista FROM producto WHERE id_producto = 1; (Devuelve 5000.00)

- Sesión B:

BEGIN;

UPDATE producto SET precio_lista = 6200.00 WHERE id_producto = 1;

COMMIT;

- Sesión A:

```
SELECT precio_lista FROM producto WHERE id_producto = 1; (Devuelve 6200.00 dentro de la misma TX)

COMMIT;
```

* Qué se observó: La misma consulta dentro de la misma transacción en Sesión A devolvió dos valores distintos porque Sesión B commiteó en el medio.

* Explicación de la IA: En READ COMMITTED, cada consulta SQL obtiene una nueva instantánea (snapshot) de la base de datos al ejecutarse, viendo cambios ya confirmados por otras transacciones.

* Verificación en el motor: Se repitió configurando en Sesión A:
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
Tras el commit de Sesión B, Sesión A volvió a consultar y el precio se mantuvo inalterable en 5000.00.

* Conclusión: Explicación certera. REPEATABLE READ congela la instantánea al inicio de la transacción mediante MVCC.

Escenario 3: Lectura Fantasma (Phantom Read)

* Cómo se reprodujo:

```
- Sesión A:
BEGIN;
SELECT COUNT(*) FROM producto WHERE id_categoria = 1; (Devuelve 1)
- Sesión B:
BEGIN;
INSERT INTO producto (nombre, precio_lista, stock, id_categoria)
VALUES ('Fugazzeta Especial', 5800, 5, 1);
COMMIT;
- Sesión A:
SELECT COUNT(*) FROM producto WHERE id_categoria = 1; (Devuelve 2)
COMMIT;
```

* Qué se observó: Una consulta agregada (COUNT) varió su resultado dentro de la misma transacción tras la inserción concurrente de Sesión B.

* Explicación de la IA: En READ COMMITTED, las filas insertadas que cumplen el filtro se vuelven visibles para lecturas posteriores. En PostgreSQL, tanto REPEATABLE READ como SERIALIZABLE impiden las lecturas fantasma gracias al snapshot de MVCC.

* Verificación en el motor: Se configuró Sesión A con SET TRANSACTION ISOLATION LEVEL REPEATABLE READ; y el segundo COUNT se mantuvo en 1.

* Conclusión: Confirmado en el motor PostgreSQL.

Declaración de Uso de IA - Parte 2

Campo	Completar
Herramienta	Kiro Agent (Integrado en VS Code)
Spec o prompt utilizado	"Explicame por qué sucede la lectura no repetible, la lectura fantasma y el bloqueo por SELECT FOR UPDATE en PostgreSQL, y qué nivel de aislamiento evita cada uno."
Qué generó	Explicaciones teóricas sobre Row-Level Locking, READ COMMITTED y el uso de MVCC en niveles superiores.
Qué se aceptó	Los conceptos teóricos de aislamiento y bloqueos.
Qué se modificó o descartó	Se ajustó el formato para integrarlo al informe de escenarios.
Verificación realizada	Se forzaron los escenarios en dos sesiones reales concurrentes de PostgreSQL modificando el isolation level y se comprobó que el motor reaccionaba tal cual explicó la IA.

Parte 3 — Riesgo Fundacional y Lectura Crítica

Ejercicio de Lectura Crítica - Parte 3

Análisis del Script 1

Script Original

```
UPDATE funcion
```

```
SET activa = FALSE;
```

* Qué filas afectaría realmente: Afecta a todas las filas de la tabla funcion sin excepción.

* Por qué no coincide con la consigna: La consigna pedía dar de baja solo las funciones de películas retiradas. Al no tener cláusula WHERE, desactiva todas las funciones del cine.

* Versión Corregida:

```
UPDATE funcion
```

```
SET activa = FALSE
```

```
WHERE pelicula_id IN (
```

```
    SELECT id
```

```
    FROM pelicula
```

```
    WHERE estado = 'RETIRADA'
```

```
);
```

```
---
```

Análisis del Script 2

```
### Script Original
DELETE FROM categoria
WHERE id NOT IN (SELECT categoria_id FROM producto);

* Qué filas afectaría realmente: Si hay algún valor NULL en
categoria_id de la tabla producto, la condición NOT IN evalúa a UNKNOWN
y no borra ninguna fila (0 filas afectadas).
* Por qué no coincide con la consigna: No garantiza borrar las
categorías huérfanas si existen nulos en la columna foránea.
* Versión Corregida:
DELETE FROM categoria c
WHERE NOT EXISTS (
    SELECT 1
    FROM producto p
    WHERE p.categoria_id = c.id
);
```

Declaración de Uso de IA - Parte 3

Campo	Completar
Herramienta	Kiro Agent (Integrado en VS Code)
Spec o prompt utilizado	"Analizó estos dos scripts SQL defectuosos (un UPDATE sin WHERE y un DELETE con NOT IN). Identificó el efecto real y reescriblos corregidos."
Qué generó	El análisis del impacto total del UPDATE y la vulnerabilidad ante valores NULL del NOT IN, junto con las sentencias corregidas usando subconsultas y NOT EXISTS.
Qué se aceptó	La explicación técnica sobre cómo SQL evalúa NOT IN con valores NULL (UNKNOWN).
Qué se modificó o descartó	Se estructuró la respuesta para separar claramente el efecto real, el motivo del fallo y la versión final.
Verificación realizada	Se leyó detenidamente la lógica del NOT EXISTS propuesta para comprobar que resolvía la falla original.